

# Git & GitHub Complete Master SOP & Rules Guide (हिंदी / Hinglish)

Push, Pull, Fetch, Branching, Merging, Stashing और Conflict Resolution के संपूर्ण प्रैक्टिकल नियम

[PUSH & PULL](#)[FETCH VS PULL](#)[BRANCH & MERGE](#)[GIT STASH](#)[CONFLICT SOLVE](#)[MULTI-ACCOUNT SAFE](#)

## 1. शुरुआत से रिपॉजिटरी सेटअप (Initial Setup from Scratch)

जब भी कोई नया प्रोजेक्ट पहली बार Git में जोड़ना हो:

```
# 1. प्रोजेक्ट फोल्डर में जाएँ: cd /d E:\Git\PCAccess # 2. गिट शुरू करें और लोकल पहचान सेट करें: git init git config user.name "Visibit Technologies" git config user.email "visibittechnologies@gmail.com" # 3. .gitignore बनाएँ (bin, obj, packages, .vs, *.bak छोड़ें) # 4. पहली बार स्टेज और कमिट करें: git branch -M main git add . git commit -m "Initial commit: Project full codebase" # 5. Remote Origin जोड़ें (Multi-account fix: URL में username प्रीफिक्स जरूर डालें): git remote add origin https://visibittechnologies@github.com/visibittechnologies/PCAccess-.git git push -u origin main
```

## 2. Fetch vs Pull के नियम (Deep Understanding)

अक्सर डेवलपर्स `fetch` और `pull` में भ्रमित होते हैं। दोनों का अंतर समझना सुरक्षित काम के लिए अनिवार्य है:

| कमांड                                      | यह क्या करता है?                                                                                                  | कब इस्तेमाल करें?                                                           |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>git fetch origin</code>              | गिटहब से सभी नई ब्रांचेस और कमिट्स की जानकारी डाउनलोड करता है, लेकिन <b>लोकल फाइलों में कोई बदलाव नहीं करता</b> । | जब आप सुरक्षित रहकर देखना चाहते हैं कि रिमोट पर दूसरों ने क्या काम किया है। |
| <code>git pull origin main</code>          | <code>fetch + merge</code> का कॉम्बिनेशन है। गिटहब से नया कोड लाता है और तुरंत <b>लोकल कोड में मिला देता है</b> । | जब आप अपने कोड को रिमोट के साथ तुरंत अपडेट करना चाहते हैं।                  |
| <code>git pull --rebase origin main</code> | गिटहब से नया कोड लाता है और आपके लोकल कमिट्स को उसके ऊपर साफ तरीके से रख देता है (बगैर फालतू Merge Commit बनाए)।  | <b>(Recommended)</b> साफ गिट हिस्ट्री बनाए रखने के लिए।                     |

```
# सुरक्षित वर्कफ्लो: पहले Fetch करके अंतर देखें, फिर Merge करें: git fetch origin git log HEAD..origin/main --oneline # देखें कि क्या नया आया है git merge origin/main # अब लोकल में मिलाएँ
```

### 3. Push के सभी प्रैक्टिकल नियम (Push Rules)

#### नियम 1: पहली बार पुश (First Push with Upstream)

```
git push -u origin main
```

-u (या --set-upstream) लगाने से लोकल ब्रांच रिमोट की ब्रांच से लिंक हो जाती है। इसके बाद आपको सिर्फ `git push` लिखना होगा।

#### नियम 2: सामान्य पुश (Daily Routine)

```
git add . git commit -m "Add: User login drive mounting feature" git push
```

#### नियम 3: किसी विशेष नई ब्रांच को पुश करना

```
git push -u origin feature-reporting
```

#### नियम 4: Push Reject होने पर क्या करें? (Never Force Push Blindly)

अगर गिटहब पर नया कोड है जो आपके पास नहीं है, तो Git कहेगा: "Updates were rejected because the remote contains work that you do not have locally."

- ❌ **कभी न करें:** `git push -f` (Force push) कभी न चलाएं! इससे दूसरों का कोड हमेशा के लिए डिलीट हो सकता है।
- ✅ **सही तरीका:** पहले `git pull --rebase origin main` चलाएं, फिर `git push` करें।

### 4. Branching & Merging (ब्रांच बनाने और मिलाने के नियम)

किसी भी नए फीचर या बग-फिक्स पर काम करते समय `main` ब्रांच में डायरेक्ट काम करने के बजाय नई ब्रांच बनाएं:

#### Step A: नई ब्रांच बनाना और स्विच करना

```
# नई ब्रांच बनाएं और तुरंत उसमें चले जाएं: git checkout -b feature-user-agent # (या आधुनिक गिट में: git switch -c feature-user-agent)
```

#### Step B: ब्रांच में काम करके कमिट करना

```
git add . git commit -m "Implement UserLoginAgent status check command"
```

#### Step C: फीचर ब्रांच को Main में मिलाना (Merge Flow)

```
# 1. मुख्य ब्रांच (main) पर वापस आएं: git checkout main # 2. सुनिश्चित करें कि main सबसे अपडेटेड है: git pull origin main # 3. फीचर ब्रांच को main में मर्ज करें: git merge feature-user-agent # 4. मर्ज किया हुआ कोड GitHub पर भेजें: git push origin main
```

#### Step D: काम पूरा होने के बाद ब्रांच डिलीट करना

```
# लोकल ब्रांच डिलीट करें: git branch -d feature-user-agent # गिटहब (रिमोट) से भी ब्रांच डिलीट करें: git push origin -delete feature-user-agent
```

## 5. Git Stash: बिना कमिट किए काम को सुरक्षित छिपाना

**सिचुएशन:** आप किसी फाइल में काम कर रहे हैं (काम अभी अधूरा है), और अचानक आपको `git pull` करना है या दूसरी ब्रांच पर स्विच करना है। Git आपको रोक देगा कहेगा "Please commit or stash your changes"।

### Stash की जादुई कमांड्स:

```
# 1. अपने अधूरे काम को सुरक्षित अलमारी (Stash) में रख दें: git stash save "Work in progress: UI styling changes"
# अब आपकी वर्किंग डायरेक्टरी बिल्कुल साफ हो गई है! अब आप pull या switch कर सकते हैं: git pull origin main # 2.
अपना छिपाया हुआ अधूरा काम वापस निकालें: git stash pop # 3. सभी सेव किए गए stashes की लिस्ट देखना: git stash
list # 4. अगर अब उस छिपे हुए काम की जरूरत न हो, तो डिलीट करें: git stash drop
```

## 6. Merge Conflicts को हल करने के नियम (How to Solve Conflicts)

**Conflict कब आता है?** जब दो लोगों ने एक ही फाइल की एक ही लाइन पर अलग-अलग बदलाव किए हों, और Git खुद फैसला न ले पाए कि किसका कोड रखना है।

### Conflict दिखने पर फाइल कैसी दिखती है?

```
<<<<<<< HEAD (आपका लोकल कोड)
string apiUrl = "https://pcaccess.ourdemos.com";
=====
string apiUrl = "https://api.pcaccess.com";
>>>>>>> origin/main (दूसरों का रिमोट कोड)
```

### Conflict सॉल्व करने के 4 आसान स्टेप्स:

- फाइल को एडिटर (VS / VS Code) में खोलें:**  
आपको `<<<<<<<`, `=====`, और `>>>>>>>` मार्कर्स दिखेंगे।
- निर्णय लें:**  
आपको जो सही कोड रखना है उसे रखें, दूसरा हटा दें, और ये तीनों मार्कर्स ( `<<<`, `===`, `>>>` ) डिलीट कर दें।
- सॉल्व करने के बाद स्टेज और कमिट करें:**

```
git add . git commit -m "Fix: Resolved merge conflict in AgentConfig" git push
```

- अगर गलती हो जाए और सब कुछ पहले जैसा करना हो (Abort):**

```
git merge --abort # (अगर rebase के दौरान conflict आया हो): git rebase --abort
```

## 7. GitHub Desktop में इन सभी ऑपरेशन्स का शॉर्टकट

| ऑपरेशन                  | GitHub Desktop में कैसे करें?                                                                                                                                                           |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fetch / Pull</b>     | ऊपर दाएँ कोने में <b>"Fetch origin"</b> बटन दबाएँ। अगर नए कमिट्स होंगे, तो वही बटन <b>"Pull origin"</b> बन जाएगा। बस क्लिक करें।                                                        |
| <b>Push</b>             | कमिट करने के बाद ऊपर बटन <b>"Push origin"</b> बन जाएगा। एक क्लिक में कोड गिटहब पर चला जाएगा।                                                                                            |
| <b>Branch बनाना</b>     | ऊपर मेनू में <b>"Current branch"</b> पर क्लिक करें → <b>"New branch"</b> दबाएँ → नाम डालें और <b>"Create branch"</b> करें।                                                              |
| <b>Branch Merge</b>     | Branch मेनू → <b>"Merge into current branch..."</b> → जिस ब्रांच को मिलाना है उसे चुनकर <b>"Create a merge commit"</b> दबाएँ।                                                           |
| <b>Stash Changes</b>    | अगर बदलाव कमिट नहीं किए और ब्रांच बदलनी है, तो GitHub Desktop खुद पूछेगा: <i>"Leave my changes on [branch]"</i> या <i>"Bring my changes to [new branch]"</i> । यह ऑटो-स्टैश कर देता है। |
| <b>Conflict Resolve</b> | GitHub Desktop फाइल्स की लिस्ट दिखाएगा और बगल में <b>"Open in Visual Studio"</b> का बटन देगा। सॉल्व करके सेव करने पर हरा टिक बन जाएगा।                                                  |

## 8. अल्टीमेट Git कमांड्स मास्टर समरी (Golden Cheat Sheet)

```
# 1. स्टेटस और हिस्ट्री git status # फाइल्स की वर्तमान स्थिति git log --oneline -n 5 # अंतिम 5 कमिट्स की लिस्ट # 2.
स्टैशिंग (अधूरा काम छिपाना) git stash # काम छिपाएँ git stash pop # काम वापस निकालें # 3. पुल और फेच git fetch
origin # रिमोट अपडेट सिर्फ चेक करें git pull --rebase origin main # रिमोट अपडेट लेकर कोड साफ़ रखें # 4. ब्रांचिंग git
checkout -b new-feature # नई ब्रांच बनाएँ git checkout main # मेन ब्रांच पर वापस आएँ git merge new-feature # फीचर
को मेन में मिलाएँ # 5. पुशिंग git push origin main # गिटहब पर भेजना
```